System Architecture

Go · HTMX · Bootstrap 5 · PostgreSQL · Cloud-agnostic object storage

🧱 Technology Stack

BROWSER	Bootstrap 5	HTMX	Vanilla JS (minimal)		*No jQuery · No SPA framework · HTML-over-the-wire*
SERVER	Go (net/http)	html/template	sqlc or pgx	Background workers	*Single binary · SSR · Multi-tenant middleware*
DATA	PostgreSQL	Row-level tenancy	RLS policies		*No schema-per-tenant · tenant_id on all rows*
INFRA	Docker / K8s	S3-compatible storage	Any cloud or bare metal		*Cloud-agnostic · Storage abstraction interface*

🔄 Request Flow

Browser HTMX request → **Tenant Middleware** Resolve tenant from subdomain or header → **Auth Middleware** Session → role (public/seeker/agent/admin) → **Handler** Business logic + DB query → **Template** SSR HTML fragment or full page → **Browser** HTMX swaps target element

🏢 Multi-Tenancy Model

Strategy: Row-level tenancy

Every table carries a `tenant_id UUID` column. PostgreSQL Row Level Security (RLS) policies enforce that queries only ever see rows belonging to the current tenant — set via a session variable at connection time.

No schema-per-tenant (operational complexity), no separate databases (cost). One schema, ironclad isolation.

Tenant resolution: subdomain-first (`foxtons.platform.com`), falling back to a request header for API clients. Middleware resolves tenant early and injects it into Go context.

```
-- Enforce on every table ALTER TABLE properties ENABLE ROW LEVEL SECURITY; CREATE POLICY tenant_isolation ON properties USING (tenant_id = current_setting( 'app.current_tenant' )::uuid); -- Set per connection in Go middleware SET LOCAL "app.current_tenant" = '<uuid>';
```

📐 Core Domain Model

tenants

- PK **id** uuid
- **slug** text (subdomain)
- **name** text
- **status** enum: pending/active/suspended
- **subscription_tier** enum
- **created_at** timestamptz

branches

- PK **id** uuid
- FK **tenant_id** uuid
- **name** text
- **address** jsonb
- **phone** text
- **active** bool

users (agents + admins)

- PK **id** uuid
- FK **tenant_id** uuid
- FK **branch_id** uuid nullable
- **email** text unique
- **password_hash** text
- **role** enum: agent/agency_admin/platform_admin
- **full_name** text

properties

- **PK** **id** uuid
- **FK** **tenant_id** uuid
- **FK** **branch_id** uuid
- **FK** **agent_id** uuid
- **status** enum: draft/active/under_offer/let/archived
- **address** jsonb + PostGIS point
- **price_pcm** int (pence)
- **bedrooms, bathrooms** int
- **features** jsonb
- **description** text

availability_slots

- **PK** **id** uuid
- **FK** **tenant_id** uuid
- **FK** **property_id** uuid
- **FK** **agent_id** uuid
- **starts_at** timestamptz
- **ends_at** timestamptz
- **capacity** int (1 = private, N = open house)
- **booked_count** int

bookings

- **PK** **id** uuid
- **FK** **tenant_id** uuid
- **FK** **slot_id** uuid
- **FK** **seeker_id** uuid nullable (guest ok)
- **guest_name, guest_email, guest_phone** text (if no account)
- **status** enum: confirmed/cancelled/attended/no_show
- **confirm_token** text (for guest email link)

seekers

- **PK** **id** uuid
- **email** text unique
- **password_hash** text
- **full_name** text
- **phone** text
- **created_at** timestamptz

Note: NOT tenant-scoped (seekers are platform-wide)

messages

- **PK** **id** uuid
- **FK** **tenant_id** uuid
- **FK** **property_id** uuid
- **FK** **seeker_id** uuid
- **FK** **agent_id** uuid
- **body** text
- **sender** enum: seeker/agent
- **sent_at** timestamptz

property_media

- **PK** **id** uuid
- **FK** **tenant_id** uuid
- **FK** **property_id** uuid
- **storage_key** text (path in object store)
- **content_type** text
- **sort_order** int
- **kind** enum: photo/floorplan/video

🗺️ Route Map

Area	Method	Path	Access	Description
Public	GET	/	public	Homepage + search
	GET	/properties	public	Search results (HTMX-swappable)
	GET	/properties/:id	public	Property detail + available slots
	POST	/properties/:id/book	public	Guest or seeker booking submission
	GET	/confirm/:token	public	Guest booking confirmation + account prompt
Seekers	GET	/seekers/register	public	Registration form
	GET	/seekers/login	public	Login form
	GET	/seekers/dashboard	seeker	Saved properties, bookings, alerts
	POST	/seekers/searches	seeker	Save a search + alert
Agent Dashboard	GET	/dashboard	agent	Overview / KPI summary
	GET	/dashboard/properties	agent	Listings table (sort/filter)
	GET	/dashboard/properties/new	agent	New listing form
	GET	/dashboard/properties/:id/slots	agent	Manage availability slots
	GET	/dashboard/bookings	agent	Upcoming bookings diary
	GET	/dashboard/messages	agent	Message inbox (HTMX polling)
	GET	/dashboard/analytics	agent	Views → bookings → attended funnel
	GET	/dashboard/team	agency_admin	Branch + user management
Admin	GET	/admin/agencies	platform_admin	Agency moderation table
	GET	/admin/properties	platform_admin	Content moderation
	GET	/admin/analytics	platform_admin	Platform-wide metrics

🔐 Authentication Strategy

NOW **Session-based email/password**
Secure HTTP-only cookie session. bcrypt password hashing. Separate session stores for seekers vs agents (different tables). Auth interface abstracted so the implementation is swappable without changing handlers.

LATER **OAuth2 (Google, Microsoft)**
Auth interface already in place — add OAuth2 provider as a second implementation. Agents will primarily use Microsoft/Google; seekers will use Google. No rework of handlers required.

LATER **Enterprise SSO**
SAML or OIDC for large agency accounts. Hooks into the same auth interface.

🖼️ Image Storage Flow

1. Agent requests upload
GET /dashboard/properties/:id/media/upload-url
Server generates a pre-signed PUT URL (15 min expiry)

→

2. Browser uploads directly
Browser PUTs file directly to S3/GCS. Server is never in the upload path. No memory pressure.

→

3. Confirm & record
POST /dashboard/properties/:id/media/confirm
Server writes storage_key to property_media table.

→

4. Serve via CDN URL
Public URL pattern:
`{cdn}/{tenant_id}/{property_id}/{file}`
Storage abstraction interface: AWS S3, GCS, Azure Blob, MinIO.